

머신러닝 · 컴퓨터공학과 (4학년)

기말 프로젝트

CardioCare

종단간 머신러닝 시스템 구축

End-to-End Machine Learning System for Heart Disease Prediction

과목	기계학습

1. 개요

여러분은 CardioCare 사례 연구 — 임상 데이터로부터 심장병 발병 가능성을 예측하여 심장 전문의의 의사결정을 지원하는 시스템 — 를 통해 종단간 ML 라이프사이클 전 과정을 학습했습니다. 이번 기말 프로젝트에서는 그 시스템을 여러분이 직접 구현합니다. 01-04장에서 다룬 모든 단계를 포함합니다:

1. 문제 정의, EDA, 데이터 전처리
2. 특성 공학, 모델 학습, 실험 관리, 모델 평가
3. 테스트, 패키징(Docker), 최소한의 CI 워크플로
4. 모니터링, 데이터 드리프트 탐지, 서빙 및 재학습 전략 문서화

최종 제출물은 재현 가능한 코드 저장소(repository)와 짧은 보고서입니다. 운영 환경 수준의 시스템을 실제로 가동할 필요는 없습니다. 다만 원본 데이터에서 출발해 배포·모니터링 가능한 산출물까지 ML 모델을 책임 있게 다룰 수 있다는 것을 보여 주어야 합니다.

윤리적 관점. CardioCare는 심장 전문의의 의사결정을 보조하는 도구이며, 절대 단독으로 결정하는 시스템이 아닙니다. 보고서에서 이 점을 반드시 명시적으로 다루어야 합니다 (§5.5 참조).

2. 학습 목표

이 프로젝트를 통해 다음을 수행할 수 있음을 입증합니다:

- 체계적인 EDA 를 수행하고, 그 결과를 근거 있는 전처리 파이프라인으로 연결할 수 있다.
- 데이터 누수(leakage), 스케일링, 클래스 불균형을 고려하며 특성을 설계하고 모델을 선택할 수 있다.
- MLflow 로 실험을 재현 가능하게 추적하고, 지표에 근거하여 최종 모델을 선택·정당화할 수 있다.
- ML 파이프라인을 위한 의미 있는 단위 테스트를 작성하고, Docker 로 시스템을 패키징할 수 있다.
- Kolmogorov-Smirnov(KS) 검정으로 데이터 드리프트를 탐지하고, 안전한 재학습·피드백 전략을 제안할 수 있다.

- 위 내용을 기술적으로 분명하게 전달할 수 있으며, 시스템의 윤리적 한계도 함께 설명할 수 있다.

3. 데이터셋

UCI Heart Disease 데이터셋을 사용합니다 (고전적인 Cleveland 부분 집합을 사용해도 무방하며, 행 수가 더 많은 통합 버전 사용을 권장합니다):

<https://archive.ics.uci.edu/dataset/45/heart+disease>

큐레이션된 Kaggle 버전을 사용해도 좋습니다. 어떤 버전을 사용했는지 명시하고, 데이터를 결정론적으로 불러오거나 다운로드하는 스크립트를 제공해야 합니다. 타깃이 다중 클래스라면 이진화(target = 0 정상 / target = 1 심장병)해 사용합니다.

4. 필수 도구

강의에서 다룬 스택을 그대로 사용합니다. 라이브러리를 추가하는 것은 자유이지만, 아래 도구를 다른 것으로 대체하지는 마십시오:

- Python 3.10+, pandas, numpy, scikit-learn, matplotlib 또는 seaborn
- mlflow — 실험 추적
- unittest (표준 라이브러리) — 단위 테스트
- scipy.stats.ks_2samp — 드리프트 탐지
- logging (표준 라이브러리) — 추론 로깅
- docker (Dockerfile + 이미지 빌드)
- CI 도구 1 종 (GitHub Actions 권장) — 기본 테스트 워크플로

5. 필수 과업

총 다섯 부분이며, 반드시 순서대로 진행하십시오. 뒤의 과업은 앞의 결과물에 의존합니다.

5.1 데이터 이해 및 전처리 (1 장)

01_eda_preprocessing.ipynb 노트북 (또는 섹션이 명확하게 주석 처리된 .py)에 다음을 포함합니다:

- 데이터셋을 불러와 `head()`, `info()`, `describe()` 결과를 출력.
- `value_counts(normalize=True)`로 타깃 클래스 분포를 보고하고, 그것이 평가 지표 선택에 미치는 영향을 서술.
- 결측값(열별·전체)과 이상치(boxplot + IQR 또는 z-score)를 최소한 연속형 특성(age, trestbps, chol, thalach, oldpeak 등)에 대해 탐지.
- 결측값(컬럼별로 삭제 vs. 평균/중앙값/KNN 대치를 정당화), 이상치, 빈 컬럼, 중복을 처리하는 전처리 파이프라인 구성. 일회성 셀이 아니라 **새 데이터에 그대로 재적용 가능한** 함수 또는 `sklearn.pipeline.Pipeline` 형태여야 합니다.
- 마크다운으로 짧게: "EDA 가 무엇을 알려 주었으며, 그에 따라 전처리에서 무엇을 어떻게 바꾸었는가?"

5.2 특성 공학, 모델 학습, 실험 (2 장)

02_train.py에 다음을 포함합니다:

- `train_test_split` 으로 데이터 분할 (시드와 비율을 명시하고 선택 근거 제시).
- 모델에 맞는 스케일링 적용 (Normalizer 또는 StandardScaler — 누수 방지를 위해 **학습 데이터에만 fit**).
- 특성 선택 수행 (예: Random Forest 기반 `SelectFromModel`) 후 채택된 특성을 보고.
- 서로 다른 계열의 모델을 **최소 3 개** 학습·비교. 권장: Logistic Regression, SVC, Random Forest, 선택적으로 KNN 또는 XGBoost.
- 모든 실행(run)에 대해 MLflow 로 파라미터, 아래의 지표 일체, 학습된 모델 아티팩트, 모델 계열 태그를 기록.
- 5-fold 교차 검증 및 가장 유력한 후보 모델에 대한 하이퍼파라미터 탐색(그리드 또는 랜덤) 최소 1 회 수행.
- 모든 모델에 대해 balanced accuracy, precision, recall, F1, confusion matrix 를 보고.
- 최종 모델을 선택하고 임상적 맥락에서 3~5 문장으로 선택 사유를 정당화. **False Negative**(1 장 토론 Q1)에 특히 유의하십시오.

5.3 테스트 및 패키징 (3 장)

다음은 제출합니다:

tests/ 폴더에 unittest 테스트 메서드 **최소 4개** 포함:

1. 예측 결과의 shape 가 입력 shape 와 일치하는지.
 2. 예측 확률이 [0, 1] 범위 내에 있고 행마다 합이 약 1 인지.
 3. 임상적으로 범위가 정해진 특성(예: chol 이 [0, 600])에 대한 입력값 범위 검증.
 4. 고정 시드에서 파이프라인이 결정론적인지 (동일 입력 → 동일 출력).
- Dockerfile — requirements.txt 기반 의존성 설치, 코드 및 저장된 모델 복사, 추론 엔트리포인트 실행. `docker build -t cardiocare:1.0` 로 빌드되고, 작은 배치 입력 파일로 정상 실행되어야 합니다.
 - .github/workflows/ci.yml (또는 동등한 설정) — 모든 push 에서 unittest 가 실행되어야 함. main 브랜치의 CI 통과(green) 도 채점 대상입니다.
 - 짧은 서술: 여러분의 파이프라인에서 **피쳐 스토어에 들어가야 할 피쳐 하나와 모델 레지스트리에 기록해야 할 메타데이터 하나**를 들고, 그 이유를 설명. 실제로 Feast 나 MLflow Registry 를 배포할 필요는 없으며, 논리만 분명히 제시하면 됩니다.

5.4 모니터링 및 드리프트 (4 장)

04_monitor.py에 다음을 포함합니다:

- 추론 경로에 logging 기반 계측 추가: 타임스탬프, 모델 버전, 입력 shape, 예측값, (가능한 경우) 실제 정답. 파일로 로그를 남겨야 합니다.
- 테스트 셋 복사본에서 연속형 특성 최소 하나의 분포를 인위적으로 이동(shift) — 예: chol 평균을 +30, 분산을 증가.
- 각 연속형 특성에 대해 학습 분포 vs. 이동된 분포로 ks_2samp 수행, p-value 보고, $p < 0.05$ 인 특성을 플래그.
- 원본 테스트셋의 balanced accuracy vs. 드리프트된 테스트셋의 balanced accuracy 를 비교 보고하여, 입력 드리프트와 성능 저하의 연관성을 가시화.
- 시간에 따른 지표 변화를 보여 주는 선택 지표의 시계열 그래프 포함 (합성 타임스탬프 사용 가능).

5.5 최종 보고서 (PDF, 6–10 쪽)

report.pdf 단일 파일로 제출하며, 다음 섹션을 포함합니다:

1. 문제 정의와 사용 목적 (반복: 알리되, 결정하지 않는다 — inform, not decide).
2. EDA 핵심 결과 (그림은 2–3 개로 제한).

3. EDA 결과에 근거한 전처리 결정.
4. MLflow 실험 기반 모델 비교 표 및 최종 선택 정당화.
5. 테스트와 패키징 — 무엇을, 왜 테스트했는가.
6. 드리프트 결과 및 재학습 / 피드백 루프 계획. 폭주하는 피드백 루프의 위험성과 사람(Human-in-the-loop)이 개입할 지점을 명시.
7. 서빙 선택: Model-as-a-Service vs. On-Device 중 하나를 선택하고, 지연 시간·개인정보(PHI)·업데이트 주기 관점에서 정당화.
8. 한계, 윤리적 고려 사항, 1 주가 더 주어진다면 무엇을 할 것인가.

6. 제출 체크리스트

GitHub 링크를 제출합니다 (public, 읽을 수 없으면 0점 처리):

```
.
├── data/           # 또는 데이터를 받아오는 스크립트
├── notebooks/
│   └── 01_eda_preprocessing.ipynb
├── src/
│   ├── preprocessing.py
│   ├── train.py
│   ├── inference.py
│   └── monitor.py
├── tests/
│   └── test_pipeline.py
├── mlruns/         # MLflow 아티팩트 (용량이 크면 스크린샷 대체)
├── Dockerfile
├── requirements.txt
├── .github/workflows/ci.yml
├── report.pdf
└── README.md       # 전체 재현 절차
```

README.md만 따라가면 채점자가 다음 정도의 단계로 전 과정을 재현할 수 있어야 합니다:

clone → pip install -r requirements.txt → python src/train.py → docker build → pytest 또는 python -m unittest.

7. 권장 순서

순서	집중할 작업	산출물
1	EDA + 전처리 파이프라인	\$5.1 노트북

순서	집중할 작업	산출물
2	특성 공학 + 모델 3개 MLflow 기록	§5.2 v1
3	교차 검증 + 하이퍼파라미터 튜닝 + 최종 모델 결정	§5.2 완료
4	단위 테스트 + Dockerfile 빌드·실행 확인	§5.3 일부
5	CI 통과(green) + 모니터링 + 드리프트 시뮬레이션	§5.3, §5.4
6	보고서 초안, 그림, 재학습 전략 작성	§5.5 초안
7	정리, 깨끗한 환경에서 재현성 검증, 제출	전체

8. 기본 원칙

- **개인 과제.** 개념에 대한 토론은 가능하나, 코드·실험 결과·보고서는 본인의 것이어야 합니다. 문서·튜토리얼에서 차용한 코드는 출처를 표기하십시오.
- **AI 도구**(Copilot, ChatGPT, Claude 등)는 보일러플레이트 작성과 디버깅에 한해 사용 가능하나, 사용 방식을 보고서 부록에 한 단락으로 **반드시 공개**해야 합니다. 제출한 모든 코드에 대해 본인이 책임지며, 구두 확인을 받을 수 있습니다.
- **데이터 누수 금지.** 데이터로부터 학습되는 모든 변환(스케일러, 임퓨터, 특성 선택기 등)은 반드시 학습용 fold에만 fit 해야 합니다.
- **재현성도 채점 대상입니다.** 랜덤 시드를 고정하고, requirements.txt 에 버전을 고정하십시오.

평가 루브릭 (총 100 점)

각 행마다 네 단계의 성취 수준이 있습니다. 채점자가 각 행에 대해 하나의 단계를 선택하고, 그 단계의 점수를 합산합니다.

평가 항목	우수 (만점)	양호 (≈80%)	보통 (≈60%)	미흡 (≈30%)	배 점
EDA · 데이터 이해 (§5.1)	체계적이고 정돈된 EDA: 클래스 분포 ·결측치·이상치·분포 모두를 적절한 시각화와 함께 검토하고, 전처리 결정으로 명확히 연결되는 통찰을 글로 정리.	필수 검토 항목 모두 포함; 통찰이 정리되어 있으나 전처리와의 연결이 일부 약함.	기계적인 EDA(head/info 수준); 일부 검토 누락 또는 해석 부실.	형식적이거나 누락; EDA와 이후 전처리 사이의 연결 없음.	10
전처리 파이프라인 (§5.1)	재실행 가능한 파이프라인(함수 또는 sklearn.Pipeline); 컬럼별 삭제 vs. 대체 결정이 정당화됨; 중복·빈 컬럼·이상치 처리; 누수 없음.	누수 없이 전 과정이 실행됨; 일부 결정의 근거가 약함.	동작하지만 파이프라인이 아닌 임시 셀로 구성; 누수 위험 존재.	명백한 누수, 깨진 파이프라인, 여러 필수 단계 누락.	10
특성 공학 · 선택 (§5.2)	모델별로 스케일링 선택이 정당화됨; 특성 선택을 분할 이후에 수행; 선택된 특성 셋 보고 및 논의 포함.	필수 단계 모두 수행; 일부 근거 부족.	스케일링 또는 선택을 수행했으나 누수 또는 정당화 부족.	누락, 분할 전에 전체 데이터에 적용, 또는 명백히 잘못된 수행.	10
모델링 · MLflow 실험 (§5.2)	3개 이상 모델 계열 비교; 모든 실행에 대해 파라미터·지표·아티팩트가 MLflow에 기록됨; CV 및 하이퍼파라미터 탐색 문서화; 정돈된 비교 표 제공.	모든 모델 학습 및 기록 수행; CV 또는 튜닝	모델은 학습되었으나 MLflow 기록이 부분적·일관성 부족; 튜닝 없음.	단일 모델, 추적 없음, CV 없음.	15

평가 항목	우수 (만점)	양호 (≈80%)	보통 (≈60%)	미흡 (≈30%)	배 점
		닝이 단 순한 수 준에서 존재.			
평가 · 모델 선택 (§5.2)	balanced accuracy-precision-recall-F1-confusion matrix 모두 보고; 임상적 맥락(특히 false negative)을 반영한 지표 기반의 최종 선택 정당화.	모든 지표 보고; 정당화는 합리적이거나 임상적 시각이 약함.	accuracy만 보고하거나 confusion matrix 누락; 정당화 부실.	지표 근거 없이 모델 선택.	10
테스트 (§5.3)	shape·확률 범위·입력 범위·결정론을 포함한 의미 있는 unittest 4개 이상; 사소하지 않은 실제 버그를 잡아낼 수 있는 테스트.	테스트 4개 존재; 1~2개는 깊이가 얇음.	테스트 4개 미만 또는 자명한 명제만을 검증.	누락 또는 동작하지 않음.	8
Dockerization (§5.3)	docker build 성공; 컨테이너가 샘플 입력으로 추론을 정상 수행; 이미지가 가볍고, 비밀값이 포함되지 않음.	빌드·실행 가능; 불필요한 비대화 또는 사소한 위생 문제 존재.	빌드는 되지만 런타임 오류 또는 워크플로가 불명확.	빌드 실패 또는 Dockerfile 누락.	7
CI 워크플로 (§5.3)	CI 설정 존재, push 시 실행, main 브랜치 green 상태; CI에서 테스트가 실제로 실행됨.	CI가 존재하고 테스트를 실행하나 빨간 실행이 남아 있음.	CI가 존재하지만 테스트를 의미 있게 실행하지 않음.	CI 설정 없음.	5

평가 항목	우수 (만점)	양호 (≈80%)	보통 (≈60%)	미흡 (≈30%)	배 점
모니터링 · 드리프트 탐지 (§5.4)	모델 버전·예측값·정답이 포함된 로깅; 연속형 특성에 대해 KS 검정 올바르게 적용; 드리프트된 셋에서 플래그가 올바르게 표시됨; 성능 저하가 나란히 제시됨.	로깅·KS 검정 모두 올바르게 구현; 성능 비교가 약함.	KS 검정은 수행했으나 해석에 결함이 있거나(예: 비연속형에 적용) 로깅이 형식적.	누락 또는 근본적으로 잘못됨.	10
서빙 · 재학습 전략 (§5.4, §5.5)	명확한 재학습 정책(드리프트 트리거 vs. 정기 학습), Human-in-the-loop 지점 명시, 폭주 루프 위험 인식; 서빙 선택(MaaS vs. 온디바이스)을 지연·개입 정보·업데이트 주기 관점에서 정당화.	합리적인 전략 제시; 서빙 정당화 3개 기준 중 하나가 약함.	전략은 있으나 일반론에 그침; 피드백 루프 위험에 대한 고찰 없음.	누락 또는 한 줄짜리 서술.	5
보고서 품질 (§5.5)	6–10쪽, 구조가 명확, 그림에 캡션, 모든 주장에 코드 결과의 뒷받침 존재; CardioCare의 윤리적 관점(inform vs. decide)을 직접 다룸.	모든 섹션 존재; 일부 주장에 근거 부족 또는 윤리적 고려가 피상적.	섹션 누락; 캡션 없는 그림; 보고서가 메모처럼 읽힘.	누락 또는 읽기 어려움.	5
재현성 · 코드 품질 (§6)	명시된 저장소 구조 준수; 의존성 버전 고정; 시드 고정; README만으로 전 과정 재현 가능; 명명·모듈화 적절.	사소한 수정 후 실행 가능; 구조나 시드 관련 사소한 문제.	재현하려면 손이 많이 감.	재현 불가.	5
합계					100

자동 감점

- **-10** README 가 없거나 채점자가 파이프라인을 재현할 수 없음.
- **-10** 학습/평가 파이프라인에서 데이터 누수가 발견됨.
- **-10** AI 도구 사용을 공개하지 않거나 출처 표기 없이 코드를 복사함.
- **-5** 저장소가 명시된 구조를 따르지 않음.

부록: "완료"의 기준 (학생용)

제출 시점에 채점자는 다음을 수행할 수 있어야 합니다:

1. report.pdf 를 열고 무엇을, 왜 만들었는지 즉시 이해.
2. 저장소를 clone 하고, 의존성을 설치한 뒤 한 줄 명령으로 학습 실행.
3. MLflow 를 열어 3 개 이상의 실행과 기록된 지표·아티팩트를 확인.
4. `python -m unittest` 로 테스트가 모두 통과하는 것을 확인.
5. `docker build` 와 `docker run` 으로 샘플 입력에 대한 예측을 받음.
6. `python src/monitor.py` 실행 시 드리프트가 플래그되고, 그에 상응하는 정확도 하락이 보고됨.

이 여섯 가지가 모두 참이라면, 여러분은 종단간 ML 시스템을 완성한 것입니다.